Placement Portal System

MAD1 Project Report

1. Author

Name: Siddharth Sahu
Roll Number: 22f3002567
Email: 22f3002567@ds.study.iitm.ac.in
About Me:
My name is Siddharth Sahu, a Diploma-level student in the IIT Madras BS Degree program. I am from Prayagraj and I enjoy building practical, real-world web applications and learning backend development concepts. This project helped me understand database design, authentication systems, and role-based web applications using Flask and Much more..

---

2. AI / LLM Usage Declaration

I used ChatGPT for approximately **5–10%** of the project work.
AI was used only for idea clarification, debugging help, and improving understanding of certain implementation steps.
All core logic including **database design, routing, application workflow, and implementation** was written and implemented manually..

---

3. Description

This project implements a **Placement Management Portal** where **Admins, Companies, and Students** interact in a structured placement workflow.

The system allows companies to create placement drives, students to apply for jobs, and administrators to monitor and manage platform activity.

AI/LLM usage percentage: **5 - 10%**, limited to conceptual help and debugging assistance.

---

4. Technologies Used

Core Technologies

- Flask ;- backend routing and application logic
- Jinja2 ;- dynamic HTML rendering
- SQLite ;- lightweight database
- SQLAlchemy ORM ;- model definitions and relationships
- Flask-Login ;- authentication and role-based access control
- Bootstrap 5 ;- responsive and clean UI
- Standard Python libraries — datetime, utilities, matplotlib(for charts) etc.

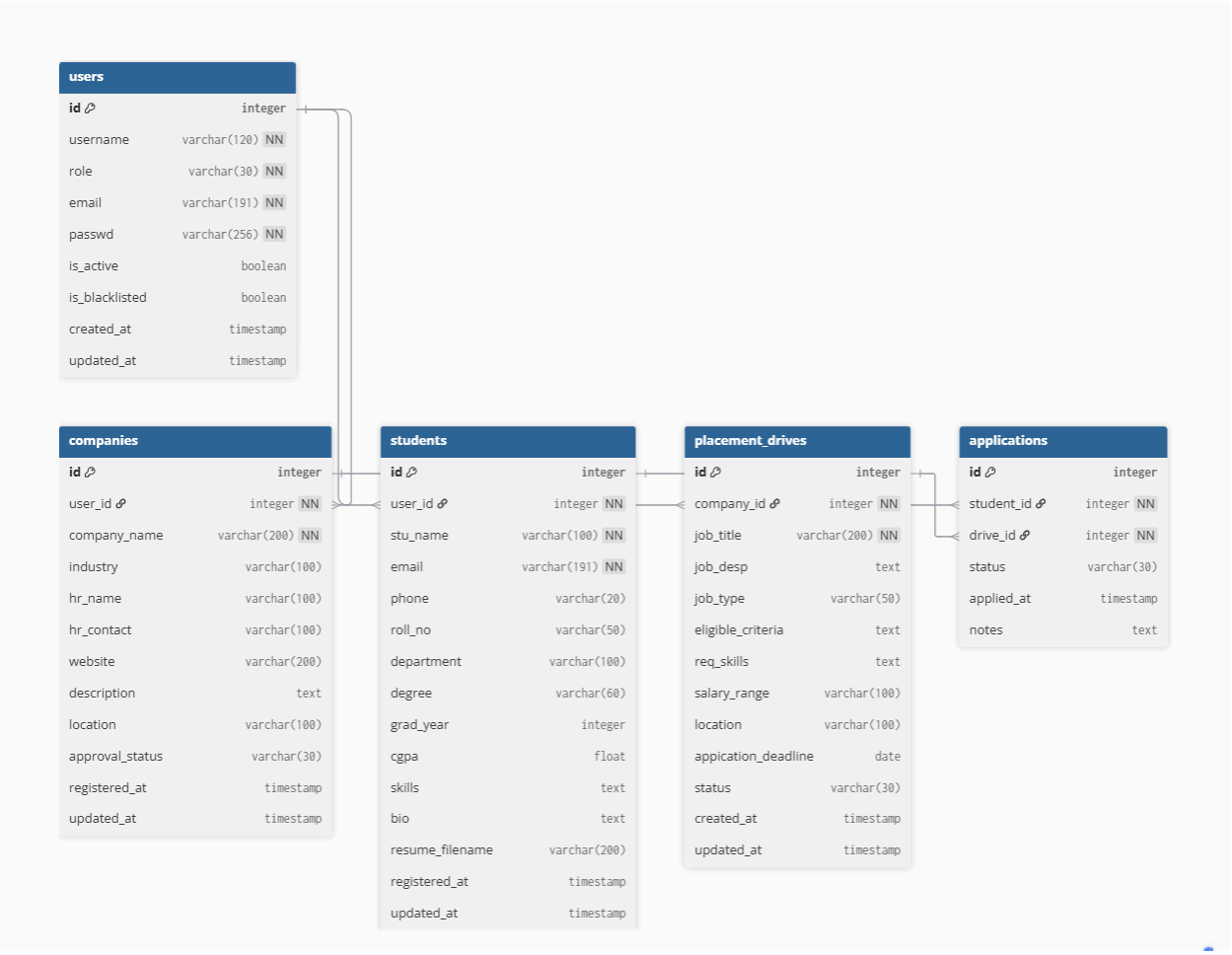
Why These Technologies?

Flask was chosen because it is lightweight and flexible for building web applications.
SQLite integrates easily without requiring external database configuration.
SQLAlchemy ORM simplifies database modeling and relationships between entities.
Bootstrap helps create a responsive and user-friendly interface.

These technologies together provide a **simple yet powerful full-stack architecture suitable for MAD-1 projects**.

---

5. Database Schema Design



Relationships

- One-to-Many → Company → PlacementDrive
- One-to-Many → Student → Application
- One-to-Many → PlacementDrive → Application
- One-to-One → User → Student / Company

Design Justification

The schema separates **authentication (User)** from **role-specific data (Student, Company)**. This improves scalability and avoids data duplication.

Placement drives and applications are connected through relational keys, allowing efficient tracking of job applications and status updates.

---

6. API Design

This project does **not implement external REST APIs**.

All functionality is implemented through **Flask routes**, which handle requests and render Jinja2 templates.

Examples of routes include:

- `/login` :- user authentication
- `/register` :- user registration
- `/student/dashboard` :- student dashboard
- `/company/create_drive` :- create placement drive
- `/apply/<drive_id>` :- apply for a job

Since MAD-1 focuses on **server-side rendering**, Flask routes are sufficient for application interaction.

---

7. Architecture and Features

Project Structure Overview

- **app.py** — main application entry point
- **models.py** — all SQLAlchemy models and DB setup
- **templates/** — HTML templates for Admin, Doctor, Patient workflows

- **static/** — Charts
- **SQLite database** is generated programmatically on first run

The project follows an MVC-like separation:

- Models represent DB structure
- Controllers are Flask routes
- Views are Jinja2 templates

Implemented Features

Admin Features

- Admin login system
- Approve or reject company registrations
- View platform statistics
- Monitor placement drives
- Blacklist or remove users

Company Features

- Register and create company profile
- Create and manage placement drives
- View student applications
- Shortlist, select, or reject candidates

Student Features

- Register and create student profile
- Complete profile information
- Browse available placement drives
- Apply to eligible drives
- Track application status

System Features

- Role-based authentication using Flask-Login
- Secure password hashing
- Automatic database creation
- Profile completeness tracking
- Clean and responsive UI using Bootstrap

---

8. Video Presentation

Video Link:
(https://drive.google.com/drive/folders/1aJlxIErW0Rulc3kbQL3vggts2s_nGKBb?usp=sharing)